

# ⊙ · Symmetry-Preserving Neural Networks

*“A molecule does not concern itself with the orientation of the coordinate axes.”*

## 1 Symmetry in physical systems

A molecule’s energy doesn’t change if you rotate it or translate it through space. This is a *symmetry*: a transformation that leaves physical quantities unchanged. In the previous chapter, we saw how CNNs encode translation symmetry and GNNs (and Deep Sets) encode permutation symmetry. Physical systems often have additional continuous symmetries—rotations, translations, reflections—that we can build into neural network architectures.

The architectures in this chapter are all GNNs at their core. They use the same message-passing framework from the previous chapter—the difference is *how* messages are constructed. Recall the general form: each node  $v$  updates its features by aggregating messages from neighbors,

$$h_v^{(\ell+1)} = \phi \left( h_v^{(\ell)}, \sum_{u \in \mathcal{N}(v)} \psi(h_v^{(\ell)}, h_u^{(\ell)}) \right) \quad (1.1)$$

where  $\psi$  is the message function and  $\phi$  is the update function. (Messages can also depend on edge features  $e_{uv}$ —e.g., bond type in a molecule—but we won’t use them here.) By restricting what geometric information enters  $\psi$  (distances only? relative vectors? higher-order features?), we control what symmetries the network respects.

## 2 Invariance and equivariance

When we apply a symmetry transformation  $g$  to the input, the output can respond in two fundamentally different ways.

*What is  $g \in G$ ?—*  $G$  is a *group*—a collection of transformations that can be composed and undone. We write  $g \in G$  for a single transformation in the group. Common groups in this chapter:

- $S_n$  — **Permutations** of  $n$  objects. GNNs handle this via summation over neighbors.
- $\text{SO}(3)$  — **Rotations** in 3D.
- $\text{O}(3)$  — **Rotations + reflections**.
- $\text{SE}(3)$  — **Rotations + translations** (no reflections).

- **E(3) — Rotations + translations + reflections.** The full Euclidean group.

Most molecular properties are E(3)-invariant (energy) or E(3)-equivariant (forces). Some quantities like chirality distinguish reflections, requiring SE(3) instead.

**Invariance.** An *invariant* function produces the same output regardless of how the input is transformed:

$$f(g \cdot x) = f(x) \quad (2.1)$$

Energy is the canonical example. Rotate a molecule by any angle, translate it anywhere in space—the energy stays exactly the same. The function “doesn’t see” the transformation.

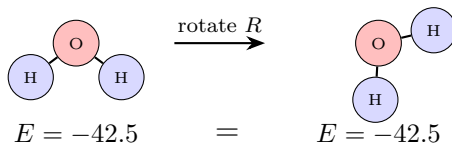
**Equivariance.** An *equivariant* function has outputs that transform consistently with the inputs:

$$f(g \cdot x) = g \cdot f(x) \quad (2.2)$$

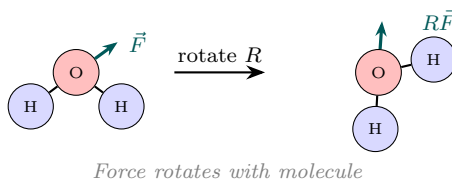
Forces are equivariant. Rotate a molecule, and the force vectors rotate with it. The force on atom  $i$  still points in the same direction *relative to the molecule*—but in the lab frame, it has rotated along with everything else. The function “sees” the transformation and responds accordingly.

The distinction matters for what you’re predicting. Scalar quantities—energy, charge, mass, binding affinity—should be invariant. Vector quantities—forces, velocities, dipole moments—should be equivariant. Per-atom outputs that describe spatial relationships (like predicted displacements) should be equivariant.

#### Invariance (Energy)



#### Equivariance (Force)



**Figure 1.** Invariance vs. equivariance. (Top) Energy is invariant: rotating the molecule doesn’t change its energy. (Bottom) Force is equivariant: rotating the molecule rotates the force vectors.

### 3 Translation invariance: use relative positions

Translation invariance is the easiest symmetry to encode: **never use absolute positions.**

If your input is atom positions  $\{r_i\}$ , don't feed raw coordinates into the network. Instead, use quantities that are intrinsically translation-invariant: *relative positions*  $r_{ij} = r_i - r_j$  between pairs of atoms, or *distances*  $d_{ij} = \|r_i - r_j\|$ . Shift all atoms by the same vector, and these quantities don't change—the relative geometry is preserved.

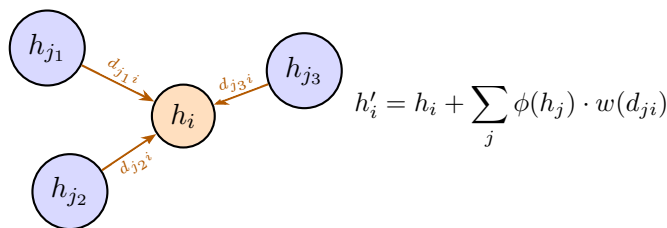
#### 4 Rotation invariance: use distances

With translation handled, we turn to rotations. The simplest approach: **only use distances**. Distances  $d_{ij} = \|x_i - x_j\|$  don't change under rotation. If your network only sees distances (never positions or relative vectors), it's automatically invariant.

**SchNet-style architecture.** Recall message passing from the previous chapter: each node  $i$  in a graph updates its features by aggregating information from neighbors  $j \in \mathcal{N}(i)$ . SchNet [1] makes this rotation-invariant by having messages depend only on distances:

$$h'_i = h_i + \sum_{j \in \mathcal{N}(i)} \phi(h_j) \cdot w(d_{ij}) \quad (4.1)$$

Each neighbor  $j$  sends a message  $\phi(h_j)$  weighted by a learned function of distance  $w(d_{ij})$ . The network never sees coordinates directly—only pairwise distances. This guarantees rotation and translation invariance. And since the sum over neighbors is order-independent, permutation invariance comes for free—just as in standard GNNs.



**Figure 2.** SchNet-style invariant message passing. Each neighbor sends a message weighted by a learned function of distance  $w(d_{ji})$ . The network only sees distances, never positions, ensuring rotation and translation invariance.

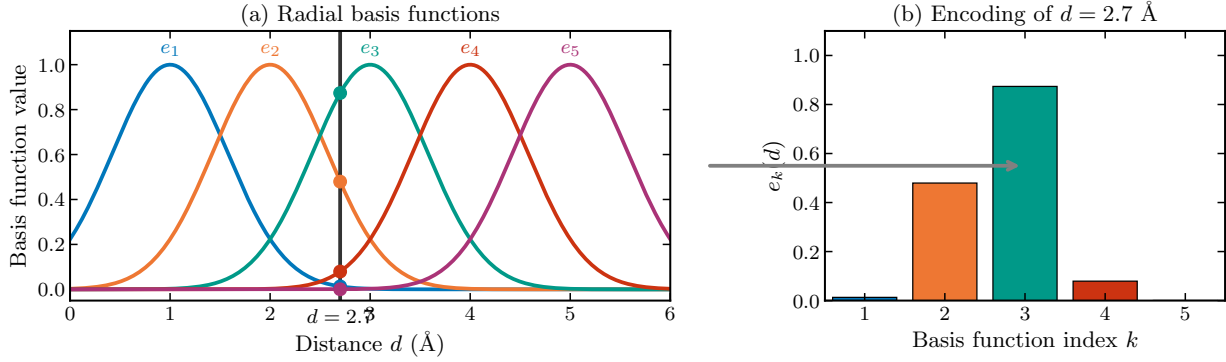
**Encoding distances.** How should  $w(d_{ij})$  depend on distance? A neural network needs a good representation of the scalar  $d_{ij}$ . The standard approach: *radial basis functions*.

Expand the distance into a set of basis functions, typically Gaussians centered at different values:

$$e_k(d) = \exp(-\gamma(d - \mu_k)^2) \quad (4.2)$$

with centers  $\mu_k$  spaced from 0 to some cutoff (e.g.,  $\mu_k = 0, 0.5, 1.0, \dots, 5.0 \text{ \AA}$ ). This gives a vector  $[e_1(d), e_2(d), \dots, e_K(d)]$  that the network can process with standard linear layers.

Why not just use  $d$  directly? A single scalar gives the network little to work with. The basis expansion provides a richer representation: each basis function “activates” for distances near its center, giving the network easy access to distance information at different scales.



**Figure 3.** Radial basis function encoding. (a) Five Gaussians centered at different distances. A vertical line marks an example distance  $d = 2.7 \text{ \AA}$ ; dots show where it intersects each basis function. (b) The resulting encoding: a 5-dimensional vector where each entry is one basis function’s activation. One scalar (distance) becomes a vector the network can process.

**Cutoff functions.** Atoms far apart don’t interact much. For efficiency (and physics), we restrict to neighbors within a cutoff radius  $r_c$ :

$$\tilde{w}(d) = w(d) \cdot f_{\text{cut}}(d) \quad (4.3)$$

where  $f_{\text{cut}}(d)$  smoothly goes to zero as  $d \rightarrow r_c$ . A common choice:

$$f_{\text{cut}}(d) = \begin{cases} \frac{1}{2} \left[ \cos\left(\frac{\pi d}{r_c}\right) + 1 \right] & d < r_c \\ 0 & d \geq r_c \end{cases} \quad (4.4)$$

Smooth cutoffs are relevant since discontinuities in the energy would cause infinite forces.

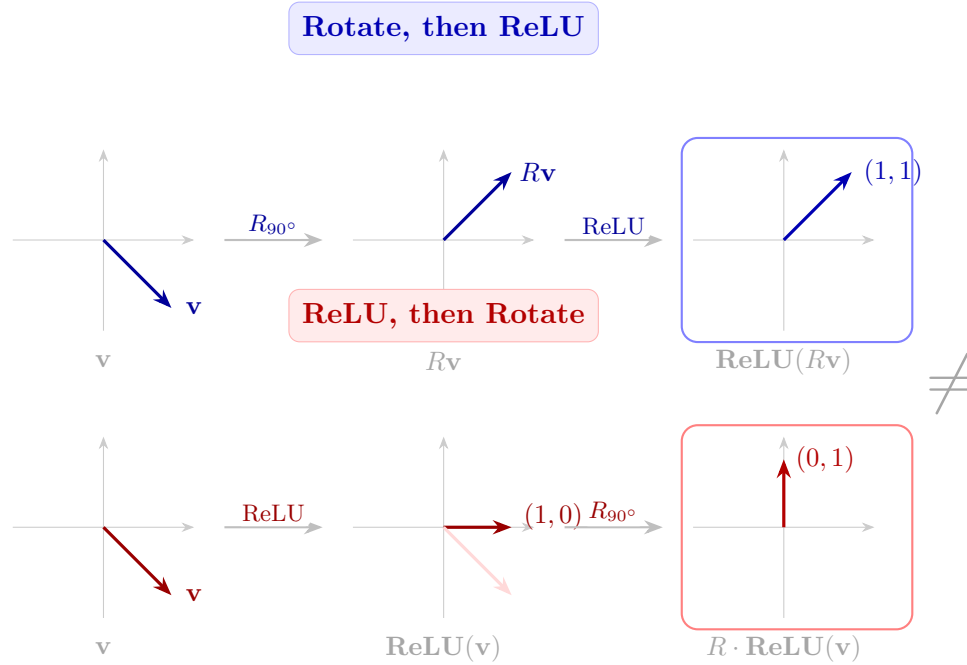
## 5 Rotation equivariance: when you need vectors

Distance-based networks are rotation-invariant—but they can only predict scalars. What if you need *vector* outputs—forces, velocities, dipole moments? These should rotate with the input, so we need *equivariance*.

**The problem with standard nonlinearities.** You can’t just apply a ReLU to each component of a vector. Consider  $\mathbf{v} = (1, -1)$  and a  $90^\circ$  rotation  $R$ . If we rotate first, we get  $R\mathbf{v} = (1, 1)$ , then  $\text{ReLU}(1, 1) = (1, 1)$ . If we apply ReLU first, we get  $\text{ReLU}(1, -1) = (1, 0)$ , then rotate to get  $(0, 1)$ . The two paths give different answers (Figure ??):  $\text{ReLU}(R\mathbf{v}) \neq R \cdot \text{ReLU}(\mathbf{v})$ . Component-wise nonlinearities break equivariance because the result depends on how the coordinate axes happen to be oriented.

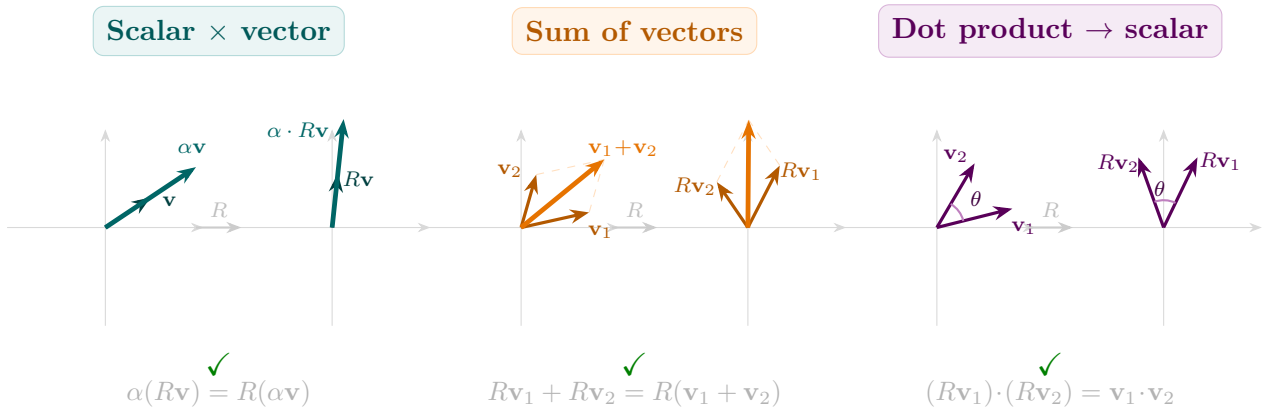
**Equivariant building blocks.** Three operations *do* commute with rotations (Figure ??):

- **Scalar  $\times$  vector.** Scaling a vector by a scalar preserves its direction, so rotation and scaling commute:  $\alpha(R\mathbf{v}) = R(\alpha\mathbf{v})$ .



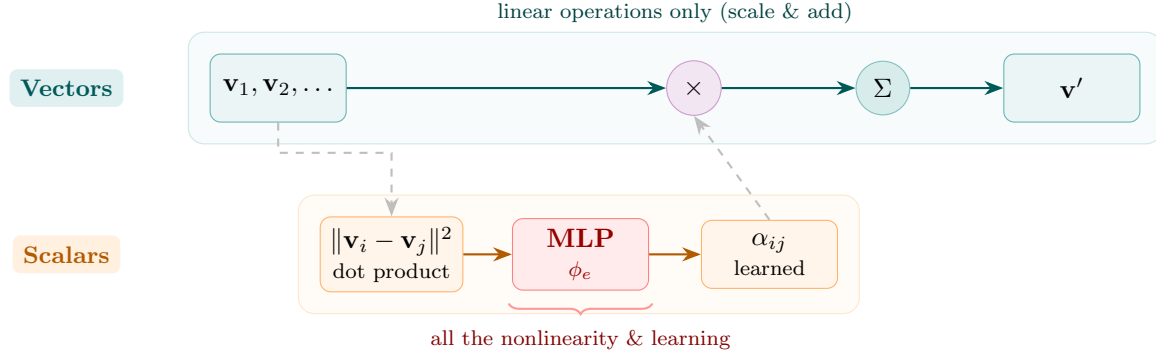
**Figure 4.** ReLU is not equivariant. Starting from the same vector  $\mathbf{v} = (1, -1)$ , the two paths—rotate then ReLU vs. ReLU then rotate—give different results. The outcome of component-wise ReLU depends on the choice of coordinates.

- **Sum of vectors.** Rotation distributes over addition:  $R\mathbf{v}_1 + R\mathbf{v}_2 = R(\mathbf{v}_1 + \mathbf{v}_2)$ .
- **Dot product  $\rightarrow$  scalar.** The dot product depends only on lengths and angles, which rotations preserve:  $(R\mathbf{v}_1) \cdot (R\mathbf{v}_2) = \mathbf{v}_1 \cdot \mathbf{v}_2$ . This produces a rotation-*invariant* scalar from two vectors.



**Figure 5.** Three operations that commute with rotations. Each panel shows the same operation applied before (left) and after (right) a rotation  $R$ —the results match. These are the only building blocks we need for equivariant layers.

**A recipe for equivariant layers.** These building blocks assemble into a general pattern (Figure ??): maintain two parallel streams—a *vector track* and a *scalar track*. The vector track stays linear: vectors can only be scaled and summed. All the nonlinearity and learning happens in the scalar track, which processes invariant quantities (like distances and dot products) through standard MLPs. The two tracks communicate via scalar-times-vector operations (scalar  $\rightarrow$  vector) and dot products (vector  $\rightarrow$  scalar). This is the template that equivariant architectures like EGNN follow.



**Figure 6.** A recipe for equivariant layers. The vector track (top, teal) uses only linear operations—scaling and summation. The scalar track (bottom, orange) handles all nonlinearity via MLPs. Dashed arrows show the crossflows: dot products send information from vectors to scalars; learned scalars modulate vectors via multiplication.

**EGNN.** EGNN [2] is a direct instantiation of this recipe. Each node has a scalar embedding  $h_i \in \mathbb{R}^d$  and a coordinate  $x_i \in \mathbb{R}^3$ . A layer updates both:

$$m_{ij} = \phi_e(h_i, h_j, \|x_i - x_j\|^2) \quad (\text{edge messages—scalar, rotation-invariant}) \quad (5.1)$$

$$x'_i = x_i + \sum_{j \neq i} (x_i - x_j) \phi_x(m_{ij}) \quad (\text{coordinate update—vector, equivariant}) \quad (5.2)$$

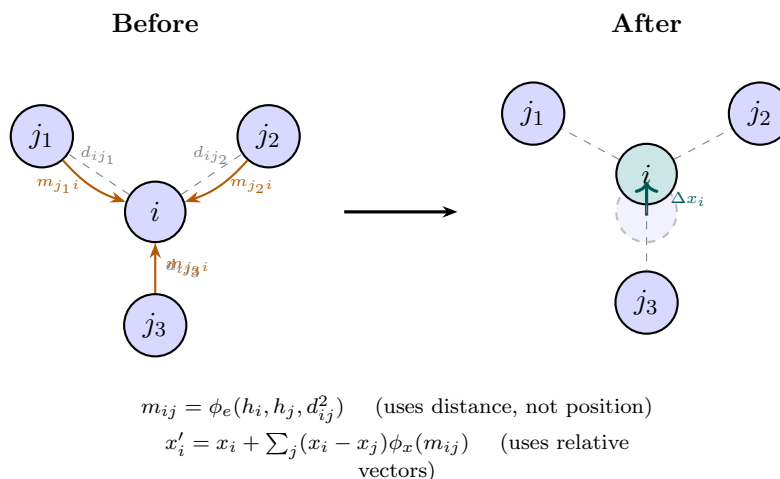
$$h'_i = \phi_h \left( h_i, \sum_j m_{ij} \right) \quad (\text{feature update—scalar, invariant}) \quad (5.3)$$

The edge messages  $m_{ij}$  depend only on *squared distances*—rotation-invariant, so they’re just scalars. The coordinate update takes the *relative position vector*  $(x_i - x_j)$  and scales it by the scalar  $\phi_x(m_{ij})$ . This is exactly the pattern above: scalar times vector gives a vector that rotates correctly. Rotate all inputs by  $R$ , and all outputs rotate by  $R$ .

**Handling velocities.** For dynamical systems, particles have velocities in addition to positions. Velocities are vectors—they rotate with the coordinate system just like positions. The original EGNN paper [2] introduces a velocity-aware variant for learning dynamics. The key modification replaces the coordinate update with:

$$x'_i = x_i + v_i \cdot \phi_v(h_i) + \sum_{j \neq i} (x_i - x_j) \phi_x(m_{ij}) \quad (5.4)$$

where  $\phi_v$  is a learned scalar function of the node features. The velocity vector  $v_i$  contributes to the position update, scaled by a learned factor—this maintains equivariance because scalar times vector transforms correctly. The velocity itself can be updated similarly, using the same equivariant pattern as coordinates. This formulation lets EGNN learn particle dynamics (like charged particles interacting via Coulomb forces) while respecting rotational symmetry.



**Figure 7.** EGNN message passing. Scalar messages depend only on distances (rotation-invariant). Coordinate updates use relative position vectors  $(x_i - x_j)$ , which rotate with the input. The combination ensures the network is E(3)-equivariant.

*Beyond vectors: spherical harmonics.*— Scalars and vectors aren't the only geometric objects. Think about what happens when you rotate the coordinate system:

- A **scalar** (like energy) doesn't change at all.
- A **vector** (like a force) rotates with the coordinates—its three components mix together according to the rotation matrix  $R$ .
- A **quadrupole** (like the moment of inertia tensor) transforms in a more complicated way—it has five independent components that mix under rotation.

This pattern continues: there's a hierarchy of geometric objects labeled by  $\ell = 0, 1, 2, \dots$ , with  $2\ell + 1$  components each. These are the *irreducible representations* of the rotation group.

**Spherical harmonics**  $Y_\ell^m(\hat{r})$  provide a concrete basis for each level. You may know them from quantum mechanics:  $\ell = 0$  is an *s*-orbital (spherical),  $\ell = 1$  gives *p*-orbitals (three lobes along  $x, y, z$ ),  $\ell = 2$  gives *d*-orbitals (five cloverleaf patterns), and so on. They form a complete basis for any angular pattern on a sphere.

More advanced equivariant networks—NequIP [3], MACE [4], Allegro [5]—represent features as collections of spherical harmonic components at each atom. The key operation is the *tensor product*: combining two spherical harmonic features (say,  $\ell = 1$  and  $\ell = 1$ ) produces

features at multiple orders ( $\ell = 0, 1, 2$ ). The coefficients governing this combination are called *Clebsch-Gordan coefficients*—fixed numbers from representation theory, not learned parameters.

The payoff is these networks capture directional interactions that scalar/vector architectures miss, like the angular dependence of chemical bonds or the anisotropy of crystal environments. The cost: mathematical and computational complexity. For a comprehensive introduction, see [6].

## 6 Forces from energy: equivariance for free

Suppose you train an invariant network to predict energy  $E(\{x_i\})$ . The forces are gradients:

$$F_i = -\nabla_{x_i} E \quad (6.1)$$

If  $E$  is rotation-invariant, then  $F_i$  is automatically rotation-equivariant.

Working through it: Let  $R$  be a rotation matrix. Invariance means  $E(R\{x_i\}) = E(\{x_i\})$ . Differentiate both sides with respect to  $x_i$ :

$$R^\top \nabla_{Rx_i} E = \nabla_{x_i} E \quad (6.2)$$

So  $\nabla_{Rx_i} E = R \nabla_{x_i} E$ . The gradient at the rotated position equals the rotated gradient at the original position. Equivariance!

The practical payoff is that we can train a simple invariant network on energies and get forces for free by automatic differentiation. The forces are then exactly equivariant. This is how most modern machine learning potentials work: learn  $E$ , differentiate to get  $F$ , run molecular dynamics. More generally: **symmetries of a function imply symmetries of its derivatives**. Build invariance into your energy predictor, and equivariance of forces follows automatically.

## 7 Why encode symmetry?

Why go through the trouble of building symmetry into architectures? After all, neural networks are universal approximators—with enough data, shouldn't they learn any symmetry implicitly? The advantage of encoding symmetry is largest with smaller datasets (less data to learn the symmetry implicitly), larger symmetry groups (more transformations to cover by augmentation), and stricter symmetry requirements (when approximate invariance isn't good enough—e.g., forces must be exactly equivariant for stable molecular dynamics).

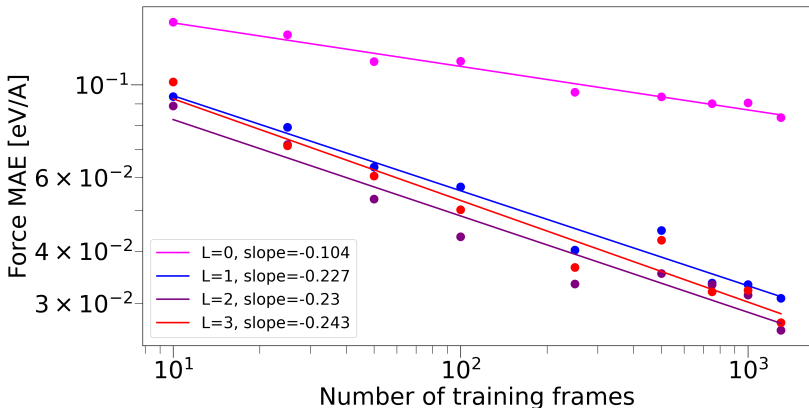
**Data augmentation: the alternative.** Instead of encoding symmetry in the architecture, we can *augment* the training data: for each molecule, include rotated copies. The network sees all orientations and (hopefully) learns that orientation doesn't matter.

This works, but has costs—training on  $N$  rotations multiplies compute by  $N$ . The network learns *approximate* invariance.



**Data efficiency.** An equivariant network doesn’t need to see every rotation of every molecule—it *knows* that rotations preserve energy. A standard network must learn this from augmented data, requiring many more examples. When labeled data is scarce (common in science, where each label may require expensive simulations or experiments), equivariance can be the difference between a useful model and an overfit one.

Figure 5 illustrated this: equivariant networks (using higher-order spherical harmonic features,  $L \geq 1$ ) achieve lower error at every training set size, with steeper improvement as data increases (more favorable scaling).

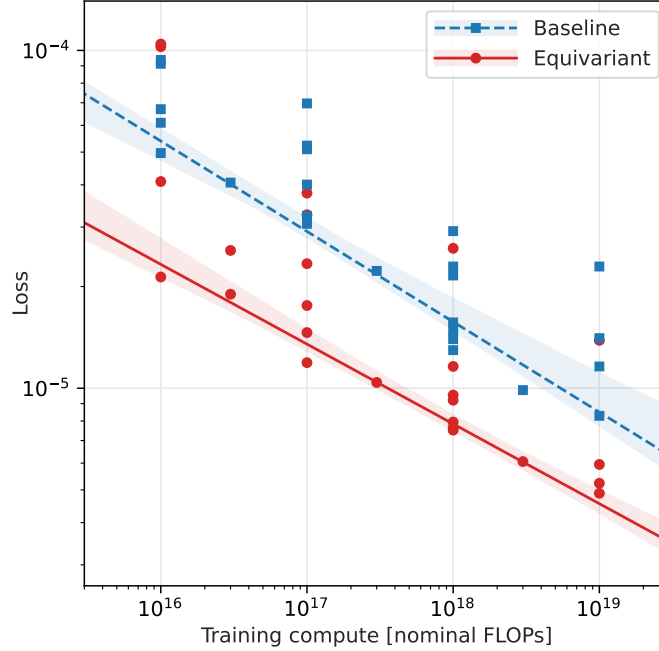


**Figure 8.** Data efficiency of equivariant networks. Force prediction error vs. training set size for water molecules.  $L = 0$  uses only scalars (invariant);  $L \geq 1$  includes vector and higher-order equivariant features. Equivariant networks achieve lower error at all training set sizes. Figure from [3].

**Can augmentation catch up?** With enough epochs, yes—but at a cost. Recent scaling studies [7] show that training non-equivariant networks with data augmentation can eventually match equivariant performance, but only when training for many more epochs on the same data. In the small-data regime where you’d train for thousands of epochs anyway, augmentation closes the gap. In the large-data regime where each sample is seen only once or a few times, equivariant networks maintain their advantage.

**Compute efficiency.** Even with infinite data, equivariant networks remain more efficient. At any fixed compute budget—measured in floating-point operations—equivariant models outperform non-equivariant ones trained with augmentation (Figure 6). Both model classes follow power-law scaling: double your compute, and the loss decreases by a predictable factor. But equivariant models maintain a consistent advantage across all tested compute budgets—roughly a factor of two in this benchmark. The symmetry constraint focuses learning capacity on the degrees of freedom that matter, rather than wasting capacity re-learning that rotations preserve energy.

**When augmentation wins.** If the symmetry is only approximate, strict equivariance may be too strong. Images have gravity—up and down aren’t equivalent. Molecules in solvent or on surfaces break full rotation symmetry. In these cases, learning an approximate symmetry from



**Figure 9.** Compute scaling of equivariant vs. non-equivariant transformers on a rigid-body dynamics task. Both follow power-law scaling, but the equivariant model (red) consistently outperforms the non-equivariant baseline (blue) at every compute budget. The advantage persists even with large-scale training. Figure from [7].

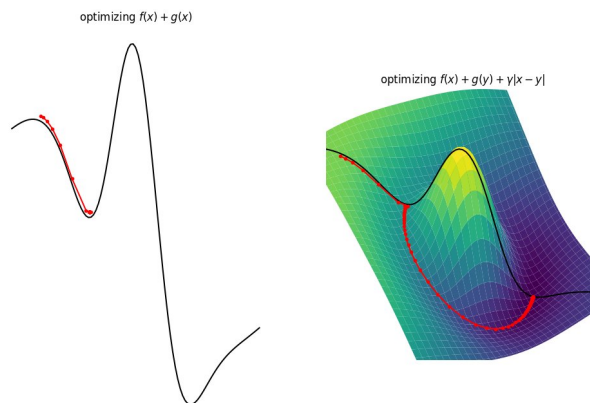
data may be more appropriate than enforcing an exact one. Augmentation also applies to any architecture without redesigning the model, which matters when leveraging pretrained models or established codebases.

**The optimization perspective.** There’s another, subtler reason constraints can hurt: they can make the *optimization landscape* harder. Building symmetry into the architecture restricts the network’s parameters to a lower-dimensional subspace—only symmetry-respecting functions are reachable. This is great for generalization (fewer parameters, no wasted capacity), but the loss landscape on that constrained subspace can have barriers that don’t exist in the full, unconstrained space.

Figure 7 illustrates this with a toy example. On the left, we optimize a function  $f(x) + g(x)$  directly: the optimizer rolls downhill into a local minimum and gets stuck behind a barrier. On the right, we *relax* the constraint by introducing a second variable: instead of requiring  $g$  to be evaluated at the same point as  $f$ , we optimize  $f(x) + g(y) + \gamma(x - y)^2$ . The penalty  $\gamma(x - y)^2$  encourages  $x \approx y$  (so the solution is close to the constrained one), but the optimizer is now free to move  $x$  and  $y$  independently. It can route around the barrier—moving  $x$  past it while keeping  $y$  on the easy side—then bring both variables together at the global minimum.

The analogy to symmetry: an equivariant architecture is the constrained problem (left). The network can only represent symmetry-respecting functions, and must optimize within that subspace. Data augmentation is the relaxed problem (right). The network has full freedom and the

augmentation loss acts as a soft penalty encouraging symmetric behavior, like the  $\gamma(x - y)^2$  term. The unconstrained landscape can be smoother and easier to optimize, even if the final solution is the same.



**Figure 10.** Constrained vs. relaxed optimization. (Left) Optimizing  $f(x) + g(x)$  directly: the optimizer gets stuck in a local minimum behind a barrier. (Right) Relaxing the problem to  $f(x) + g(y) + \gamma(x - y)^2$  lifts it into a 2D space. The black curve shows the original 1D landscape along the  $x = y$  diagonal; the red path shows how the optimizer can leave the diagonal to bypass the barrier, then return to reach the global minimum.

**Equivariance as a building block.** As emphasized throughout, the architectures in this chapter are not standalone models but composable components. E.g., diffusion models for molecules use equivariant score networks to ensure the learned distribution respects rotational symmetry, and machine learning potentials use equivariant networks to predict forces that transform correctly.

## References

- [1] Kristof T Schütt, Pieter-Jan Kindermans, Huziel E Sauceda, Stefan Chmiela, Alexandre Tkatchenko, and Klaus-Robert Müller. SchNet: A continuous-filter convolutional neural network for modeling quantum interactions. In *Advances in Neural Information Processing Systems*, volume 30, pages 992–1002, 2017.
- [2] Víctor Garcia Satorras, Emiel Hoogetboom, and Max Welling. E(n) equivariant graph neural networks. In *International Conference on Machine Learning*, pages 9323–9332. PMLR, 2021.
- [3] Simon Batzner, Albert Musaelian, Lixin Sun, Mario Geiger, Jonathan P Mailoa, Mordechai Kornbluth, Nicola Molinari, Tess E Smidt, and Boris Kozinsky. E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nature Communications*, 13(1):2453, 2022.
- [4] Ilyes Batatia, Dávid Péter Kovács, Gregor NC Simm, Christoph Ortner, and Gábor Csányi. MACE: Higher order equivariant message passing neural networks for fast and accurate force fields. In *Advances in Neural Information Processing Systems*, volume 35, pages 11423–11436, 2022.

- [5] Albert Musaelian, Simon Batzner, Anders Johansson, Lixin Sun, Cameron J Owen, Mordechai Kornbluth, and Boris Kozinsky. Learning local equivariant representations for large-scale atomistic dynamics. *Nature Communications*, 14(1):579, 2023.
- [6] Alexandre Duval, Simon V Mathis, Chaitanya K Joshi, Victor Schmidt, Santiago Miret, Fragiskos D Malliaros, Taco Cohen, Pietro Liò, Yoshua Bengio, and Michael Bronstein. A hitchhiker’s guide to geometric GNNs for 3D atomic systems. *arXiv preprint arXiv:2312.07511*, 2023.
- [7] Johann Brehmer, Sönke Behrends, Pim de Haan, and Taco Cohen. Does equivariance matter at scale? *Transactions on Machine Learning Research*, 2025.